

第 02 课

集群、数据类型、内联 PTX 与指针

Clusters, Data Types, Inline PTX, and Pointers

第 02 课 · 集群、数据类型、内联 PTX 与指针

集群、数据类型、内联

PTX、指针 Prateek Shukla

什么是线程块集群

集群是多达 16 个线程块组成的集合体，它们被保证共同并发调度在同一个 GPC 内相邻的 SM 上

这将传统 CUDA 编程层级从三层扩展为四个显式层级：线程 -> 线程块 -> 线程块集群 -> grid

集群中的所有块并发运行在物理上彼此靠近的 SM 上，从而实现高效的跨 SM 协作

使用集群的原因，大多数时候是为了把分布式共享内存用于不同的目的。

每个线程块都受限于有限的共享内存和

单个 SM 上可用的计算资源。

如果线程想要访问更多数据，它需要去全局内存取，这很昂贵

对于 top-k 或矩阵乘法这类需要访问运行在不同 SM 上的线程块所含数据的操作，线程块集群提供了一种解决方案。

它们让算法用适度的内存带宽换取跨多个 SM 大幅扩展的数据可及性

我们要做的，就是把所有线程块拿来并汇集它们的一部分内存。现在

来自不同块的线程可以访问它们需要的数据，而无需访问全局内存

这个"池"并不是说所有线程都把自己分配到的共享内存倒进同一个东西里。每个块仍然拥有自己那份共享内存，区别在于现在线程可以跨过去访问其他块的内存

通常这个池很小，并且被限制在一个 GPC 内部

几个要点

一个线程块仍然运行在单个 SM 上——这种一对一关系没有变

SM 可以同时运行多个线程块——这一直是可能的，现在依然如此

集群是软件概念，GPC 是硬件概念

没有自动分配：你必须在代码中显式定义集群；调度器不会自动把块分组

关于集群大小

集群大小为 8 个线程块时，硬件可以在每个 GPC 中容纳两个集群，高效利用每个 GPC 约 16 个 SM

值得一提的是，使用大于 8 的集群大小需要在 kernel 启动时显式设置 `cudaFuncAttributeNonPortableClusterSizeAllowed`。大小为 8 或更小时，代码保持可移植

集群大小为 16 个线程块时，每个 GPC 只能容纳一个集群，每个 GPC 约有 1-2 个 SM 闲置，整个 GPU 约有 18 个 SM 未被使用

实践发现集群大小 2 要最优得多，因为更大的集群存在隐藏的同步成本（大小 4 及以上为 87 个周期，大小 16 为 150 个周期）

什么是分布式共享内存

H100 的一项特性，允许线程块集群内的 SM 之间直接访问内存

H100 为集群实现了一张专用的 SM 到 SM 网络，提供对远程共享内存的快速、低延迟访问，使 SM 能够跨其他 SM 的共享内存执行加载、存储和原子（Atomic）操作

DSMEM 可以与 L2 缓存访问同时使用。这让应用在 SM 之间传递数据时可以利用两条路径的合并带宽

DSMEM 上的多播与 TMA

我们可以在 DSMEM 上用 TMA 做异步拷贝操作

DSMEM 最重要的特性之一是多播操作，它同时把数据投递到多个 SM 的共享内存

TMA 多播绕过了我们前面讨论的 SM 到 SM 网络瓶颈。不用线程显式地跨 DSMEM 读写（造成拥塞和同步开销），集群中单个线程就可以发起一次 TMA 多播操作，把数据一次性分发到所有 SM

创建线程块集群

你可以在 kernel 声明中使用 `__cluster_dims__` 属性在编译期定义集群维度

定义好之后你就可以正常启动 kernel，但 grid 维度必须是集群大小的倍数

集群句柄

```
#include <cooperative_groups.h> (cooperative groups 帮助管理集群)
```

```
namespace cg = cooperative_groups;
```

```
cg::cluster_group cluster = cg::this_cluster();
```

创建一个指向当前执行线程所属线程块集群的句柄 `int* remote_smem = cluster.map_shared_rank(smem, target_block_rank);`

```
remote_smem[idx] = value;
```

```
unsigned int cluster_size = cluster.num_blocks();
```

```
unsigned int cluster_rank = cluster.block_rank();
```

但我们不会经常用它。

我们一般会用 `mapa` 来把共享内存地址转换为集群中的地址

PTX 特殊寄存器

它们提供关于集群中线程块的信息

`%cluster_ctaid`: 集群内的 CTA ID

`%cluster_nctaid`: 集群的维度

`%cluster_ctarank`: CTA 在集群内的线性化 rank

`%cluster_nctarank`: 集群中 CTA 的总数

`%is_explicit_cluster`: 区分显式与隐式 (1x1x1) 集群启动

并行线程执行 (Parallel Thread Execution)

PTX 是 Nvidia GPU 的汇编语言，在 CUDA 生态中充当指令集架构 (ISA)

CUDA C++ 文件被编译时先翻译成 PTX，PTX 再经 JIT 编译成 SASS，SASS 再被汇编成二进制

我们学习 PTX 是因为它让我们能直接访问 CUDA C/C++ 中没有直接暴露的低级 GPU 特性，这将帮助我们手工优化性能关键段落

它在许多著名仓库中被广泛使用，学习 PTX 能让你了解高度优化库中的各种优化

内联 PTX

内联 PTX 是使用 `asm` 关键字直接嵌入 CUDA C/C++ 的 PTX 汇编代码

我们使用内联 PTX 是因为它省去了用 PTX 从零手写一切的大量繁重工作，并允许在 C++ 代码中编写 PTX 指令

模板能把参数化的编译期常量直接嵌入 GPU 指令，这也是我们经常把模板与内联指令一起使用的原因

PTX 指令的格式

- `asm()` 把 PTX 代码插入你的 CUDA 程序
- 在 `asm` 后面加上 `volatile` 关键字，可以防止编译器删除或移动你的 PTX 指令
- 如果你不想让编译器看到你访问内存的方式，就在 clobbers 段使用 `memory` 关键字

输入与输出操作数

输入与输出操作数是你的 C++ 变量与汇编指令之间的桥梁。它们使用基于约束的语法告诉编译器如何把变量映射到寄存器或内存

输出操作数在指令字符串之后最先声明，然后是输入操作数，最后是 clobbers

操作数先按输出再按输入的顺序连续编号

如果没有输出，你可以把字段留空，例如 `asm("string": inputs)`

约束修饰符

= —— 只写

+ —— 读写

& —— 提前 clobber (early clobber) —— 防止编译器把同一个寄存器既用于一个输出又用于后续的输入。当输出在所有输入被消费之前就要写入时，这一点至关重要

约束

h —— 16 位无符号整数

r —— 32 位无符号整数，用于 32 位地址和无符号整数

l —— 64 位无符号整数和 64 位地址

f —— 32 位浮点数

d —— 64 位浮点数

n —— 立即整数（编译期常量）

关于操作数的更多要点

我们按文本顺序表示操作数，如 %0、%1、%2

所以 %0 代表指令字符串之后的第一个变量

我们用约束修饰符和约束来表示变量（输出用 "+r"、"=f"、"+l"，输入用 "r"、"l"、"f"）

花括号用于局部寄存器作用域 {}

指令字符串

- 这是包含我们想在 GPU 上运行的实际指令的字符串
- 指令可以写在单行，也可以用换行写成多行
- 你可以使用 %0、%1 等占位符来表示操作数，它们会被冒号之后给出的汇编实际使用的操作数所替换

```
asm volatile("wgmma.mma_async {%0, %1, %2, ...}")
```

输出操作数

它们位于指令字符串之后

使用特殊修饰符和约束来表示输出

多个输出用逗号分隔表示

如果没有输出操作数，就把字段留空（如 ::）

输入操作数

位于输出操作数之后，用冒号分隔

你可以有只读的输入操作数 ("n"、"r"、"f"、"l") 或读写操作数 ("+n"、"+r"、"+f")，但不能有只写操作数

你可以用 "+n" 约束来获得编译期操作数

你可以用逗号分隔多个输入操作数，并把它们放在输出操作数之后

Clobber

它们告诉编译器，除输出操作数之外还有哪些资源可能被修改
这防止编译器做出错误假设并错误地修改代码。

CUDA 中的状态空间是什么

CUDA 中的状态空间指 GPU 线程可访问的不同内存区域，每个都为特定目的做了优化

用 PTX 写代码时，我们需要指定正在为哪个状态空间发射指令

它们不只是低级的 PTX 细节。正是因为有了它们，我们才能把内存放置当作一等决策而不是事后补救，从而写出高性能代码

寄存器状态空间 (.reg)

寄存器内存

当你需要输入/输出约束体系未覆盖的临时寄存器时，需要显式的 .reg 声明。

溢出机制：当寄存器用量超过硬件上限时，变量会自动溢出到本地内存，这可能显著影响性能。

全局状态空间 (.global)

代表全局内存

用于各种使用全局内存数据的操作

本地状态空间 (.local)

每线程私有内存：每个线程都有自己的私有本地内存空间，用于存放装不进寄存器的数据。

很少使用

参数状态空间 (.param)

双重用途：既作为 kernel 输入参数（只读，按 grid），也作为设备函数参数（读写，按线程）。

参数传递机制：用于在不占用通用寄存器堆的情况下向 kernel 和函数传递参数。

同样不常使用

共享状态空间 (.shared)

代表共享内存

集群范围可访问性：通过子限定符 (::cta 或 ::cluster)，同一集群内其他 CTA 中的线程也可以访问它，用于高级通信

数据类型

无符号整数类型 —— .u8、.u16、.u32、.u64

有符号整数类型 —— .s8、.s16、.s32、.s64

浮点类型 —— .f16、.f32、.f64

原始位模式 —— .b8、.b16、.b32、.b64、.b128

谓词 (predicate) 类型 —— 存储 TRUE 或 FALSE 值

深度学习中的的一些重要数据类型

bf16（存储为 .b16）：8 位指数，7 位尾数（共 16 位）

e4m3、e5m2（存储为 .b8）：8 位浮点格式（4 位指数 + 3 位尾数，或 5 位指数 + 2 位尾数）

tf32（存储为 .b32）—— 32 位格式，范围与 .f32 相同但精度降低（尾数 ≥ 10 位）

4 位浮点（e2m1）

超紧凑：2 位指数，1 位尾数（共 4 位）

打包数据类型

它们把多个值打包进单个寄存器以进行并行操作：

.u16x2、.s16x2 —— 在 32 位寄存器（.b32）中装两个 16 位整数

.f16x2 —— 装两个 .f16

.bf16x2 —— 装两个 BF16

.e4m3x2 —— 装两个 e4m3

.e5m2x2 —— 装两个 e5m2

内存地址到底是什么

把 GPU 的全部内存看作一把无限长的尺子。你能为其取地址的最小内存单位是字节（Byte，8 位），这也被称为原子（atom）。

地址就是距尺子起点的字节距离。字节地址（Byte Address）就是尺子上原始的整数索引

在 CUDA 中你很少直接处理原始字节。你处理的是类型（float、int、bf16）。当你写 `ptr + 1` 时，编译器移动的不是 1 字节，而是 1 个元素。

共享内存与 bank

为了让 32 个线程（一个 warp）能同时访问内存，共享内存被划分为 32 个离散的内存 bank。bank 索引从 0 到 31，每个 bank 宽 4 字节。GPU 根据 4 字节字的索引来判定一个地址属于哪个 bank。

（字节 0-3）：存于 Bank 0 （字节 4-7）：存于 Bank 1 （字节 8-11）：存于 Bank 2 （字节 12-15）：存于 Bank 3，以此类推

Bank 冲突

硬件每个时钟周期在每个 bank 上只能服务一个唯一地址。如果 2 个线程冲突，访问会被串行化（耗时 2 倍）。如果 32 个线程冲突（最坏情况），耗时 32 倍。

如果多个线程读取完全相同的地址，就没有冲突。硬件执行多播/广播，1 个周期内服务所有线程。当同一 warp 中的多个线程试图访问映射到同一 bank 的不同地址时，就会发生 bank 冲突。

H100 内存控制器以 128 字节事务为单位处理请求

如果你的 warp 总共请求超过 128 字节（例如每个线程加载一个 16 字节的 float4），该请求会被拆分为多个事务（波，wave）。

swizzle 的由来

因为 H100 依赖张量核心（它读取 64x64 的数据 tile），标准的线性寻址常常造成大规模 bank 冲突（strided 访问）。如果你有一个 64x64 的 bf16 数字 tile。行大小：64 元素 x 2 字节 = 128 字节。每行的 bank 容量：32 bank x 4 字节 = 128 字节。矩阵中的每一列都恰好对齐到同一个 bank $((128/4) \bmod 32)$ 我们要确保 `Matrix[0][0]` 和 `Matrix[1][0]` 落在不同的 bank，即使它们的步长（stride）恰好是 bank 宽度的完美整数倍

指针与状态空间

统一虚拟寻址（Unified Virtual Addressing, UVA）系统把每个状态空间映射到一个互不重叠的区域：

0x0000000000000000 -> 0x0000FFFFFFFFFFFF（保留）

0x0001000000000000 -> 0x0001FFFFFFFFFFFF（全局内存）

0x0002000000000000 -> 0x0002FFFFFFFFFFFF（每上下文的本地内存）

0x0003000000000000 -> 0x0003FFFFFFFFFFFF（每块的共享内存）

0x0004000000000000 -> 0x0004FFFFFFFFFFFF（常量内存）

泛型指针

在现代 CUDA（计算能力 ≥ 3.5 ）中，所有指针默认都是泛型的——它们是跨越所有状态空间的单一统一虚拟地址空间中的 64 位地址

没有运行时检查来判断我们处于哪个状态空间

可以像 CPU 上的指针一样做所有常规算术

当泛型指针被解引用时，硬件检查高位以把请求路由到正确的内存子系统，这在运行时空间检测上损失了几个周期

状态空间指针

当你手写内联 PTX 或编译器生成它时，指针可以被其状态空间所限定

不是指针，只是对特定硬件指令有意义的原始位

不能在 C++ 中解引用 ((float)shared_bits 会崩溃或得到垃圾值)

没有运行时空间检测，硬件立刻就知道它是共享内存

支持那些要求显式空间限定的专用指令

cvta 转换为地址

cvta（Convert To Address，转换为地址）是一条 PTX（Parallel Thread Execution）指令，用于在泛型地址与特定内存空间地址之间转换指针。

对应的 CUDA C++ API

`__cvta_generic_to_global`

`__cvta_generic_to_shared`

`__cvta_generic_to_local`

`__cvta_generic_to_constant`

我们在哪里需要它们

张量核心需要显式的共享内存地址来执行 ldmatrix

TMA 需要显式的空间限定

线程块集群需要精确的空间控制来做跨 SM 数据共享，我们使用集群专属地址来操作它们

在多实例 GPU（MIG）配置中，显式的地址空间转换有助于确保 GPU 实例之间恰当的内存隔离。

mapa 与线程块集群

用于把内存地址从 CTA 内的共享内存转换为分布式共享内存

mapa 把当前 CTA 共享内存空间中的一个共享内存地址，转换为同一集群内另一个 CTA 共享内存中的对应地址

关键点：它接受的是 rank，而不是块 ID！

rank 与块 ID

块 ID (%ctaid) : 在 grid 中的全局位置 (跨所有集群为 0 到 N-1)

rank (%cluster_ctarank) : 集群内的位置 (0 到 cluster_size-1)